Performance Testing

Date	xxxxxx
Team ID	xxxxxx
Project Name	AI Debt Relief Platform
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	User Authentication, Loan Management API, AI Debt Analysis API, Dashboard
Test Environment	Local System (React.js + Node.js + Express.js + MongoDB)
Test Date	29 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User Login API Performance	25	60	Users should log in successfully within 2 seconds
2	Add Loan API Performance	30	60	Loan details should be saved without delay
3	AI Debt Analysis API	20	90	AI should generate debt recommendations successfully

4	Dashboard Data Loading	40	60	Dashboard charts should load without errors

SMARTBRIDGE Let's Bridge the Gap

SKILL WALLET A SmartBridge Product

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.32 seconds	Pass	Response time is within acceptable limit
2	Response Time (Max)	< 5 seconds	3.68 seconds	Pass	Stable under peak load
3	Throughput (Req/sec)	50 Req/sec	72 Req/sec	Pass	Good request handling capability
4	Error Rate	< 1%	0.3%	Pass	No critical failures observed
5	CPU Utilization	< 80%	61%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	65%	Pass	Memory consumption within acceptable range

Step 4: Observations & Analysis

Key Findings

- The AI Debt Relief Platform successfully handled concurrent user requests without any system crashes.
- Average response time remained below the target value throughout testing.
- Dashboard and Loan Management modules performed efficiently under moderate load.
- AI Debt Analysis required slightly more processing time but remained within acceptable limits.

Bottlenecks Identified

- AI Debt Analysis API experienced a slight increase in response time when multiple requests were processed simultaneously.
- Database queries for loan history required optimization under higher user load.

Optimization Steps Taken

- Optimized MongoDB queries by adding indexes to frequently accessed collections.
- Reduced unnecessary API calls from the frontend using efficient state management.
- Improved backend response time through asynchronous request handling.
- Compressed API responses to reduce network latency.

Step 5: Screenshots / Evidence